

УТВЕРЖДАЮ

Заместитель директора Новосибирского
филиала – директор по прикладным проектам
ПАО «Ростелеком»

_____ П.А. Ташлыкова

« ____ » _____ 2024

МП

УТВЕРЖДАЮ

Ректор Новосибирского государственного
университета

_____ М.П. Федорук

« ____ » _____ 2024

МП

СОГЛАСОВАНО

Директор ЦИИ НГУ

_____ А.Н. Люлько

« ____ » _____ 2024

Разработка программного комплекса
для детектирования событий в медиа контенте
Программа и методика испытаний

Договор с Индустриальным партнёром на выполнение работ или оказание услуг в целях совместной реализации
ключевых мероприятий программы Центра в части внебюджетного софинансирования от 23.11.2023 г. № 0-35-2023 по
техническому заданию №1 «Разработка программного комплекса для детектирования событий в медиа контенте»

Руководитель разработки,
ведущий научный сотрудник ЦИИ НГУ

_____ А.Г. Окунев

« ____ » _____ 2024

СОДЕРЖАНИЕ

Термины, сокращения и определения.....	3
1. Объект испытаний.....	5
2. Цель испытаний.....	6
3. Требования к программе.....	6
4. Требования к программной документации.....	7
5. Средства и порядок испытаний.....	8
6. Методы испытаний системы детектирования событий.....	9
7. Результаты предварительных испытаний.....	14
8. Сценарии для испытаний детекторов.....	15

Термины, сокращения и определения

Таблица 1 – Используемые сокращения

Сокращение	Расшифровка
AP	Аналитик Провайдер
API	Application Programming Interface
ML	Machine Learning (машинное обучение)
VAS	Video Analytics Service (система видеоаналитики)
БД	База данных
ВСВН	Внешняя система видеонаблюдения
ДТП	Дорожно-транспортное происшествие
НГУ	Новосибирский Государственный Университет — Федеральное государственное автономное образовательное учреждение высшего образования «Новосибирский национальный исследовательский государственный университет»
НИОКР	Научно-исследовательские и опытно-конструкторские работы
ПО	Программное обеспечение
СВН	Средство видеонаблюдения (камера)
ТЗ	Техническое задание
ТС	Транспортное средство
Мп	Мегапиксели
ПМИ	Программа и методика испытаний

Таблица 2 – Термины и определения

Термин	Определение
Детектор	Модель распознавания целевых событий в медиа контенте, разработанная на основе алгоритмов ML.
Платформа AP	Аналитик Провайдер – платформа сервисов для взаимодействия с вендорами аналитики, система АО «Нетрис».
Smart City	«Умный город» - это созданная человеком взаимосвязанная система информационных и коммуникативных технологий с

Термин	Определение
	интернетом вещей, которая упрощает управление внутренними городскими процессами и делает жизнь жителей комфортнее и безопаснее.
baseline	Базовый продукт, на основе которого происходит сравнение или заимствование технологий.
Кроп (кадрирование)	Фрагмент изображения, акцентирующий внимание на требуемом объекте.
bbox	Bounding box, ограничивающий прямоугольник.
Компьютерное зрение	Теории и технологии создания искусственных систем, которые получают информацию из изображений.
Инференс	Процесс получения результатов от нейронной сети.
Нейронная сеть	Цепочка обработки данных, по архитектуре схожая с биологическими нейронами. Позволяет получать осмысленные выходные данные, основывающиеся на поставленной задаче.

1. Объект испытаний.

Испытаниям подвергаются ML модели детектирования (далее – детекторы) целевых событий в медиа контенте.

Данные модели реализованы с использованием функций машинного обучения, на основе алгоритмов нейросетей, либо других функций ML.

Данные детекторы предназначены для обработки медиа контента:

- видеопотока СВН в реальном времени;
- архивного видеопотока СВН;
- статических фотоизображений, являющихся скриншотами видеопотока СВН.

Испытаниям подвергаются следующие детекторы:

- Детектор лица человека.
- Детектор силуэта человека.
- Детектор лежащего человека.
- Детектор типа транспортного средства.
- Детектор определения (распознавания) государственных регистрационных знаков.
- Детектор дыма.
- Детектор огня.
- Детектор оружия.
- Детектор движения в кадре.
- Детектор резкого ускорения объекта в кадре.
- Детектор агрессивного поведения.

2. Цель испытаний

Цель проведения испытаний – проверка соответствия характеристик разработанных детекторов общим и системным требованиям, изложенным в документе Техническое задание.

3. Требования к программе

Задача оператора испытаний – убедиться, что при разработке детекторов учтены нижеперечисленные требования:

- Осуществлена разметка обучающих и тестовых изображений (датасетов).
- Программное обеспечение разработано с учетом совместимости с российскими операционными системами. Программное обеспечение использовало открытые фреймворки машинного обучения (PyTorch), ускорения инференса моделей (ONNX Runtime) и контейнеризации приложений (Docker)
- Подготовленные датасеты обеспечивают воспроизводимые тренировки и валидацию моделей, а также тестирование количественных параметров представляемых моделей.
- Обеспечена возможность интеграции каждой из разработанных моделей детектирования с Платформой АР. Время передачи результатов потребителям через API с момента вычисления результата обработки должно быть не более 5 минут.

4. Требования к программной документации

Для проведения испытаний в качестве результатов работ для каждого из разработанных детекторов должны быть предоставлены:

- исходный программный код и конфигурационные настройки разработанной модели;
- размеченные входной, тренировочный датасеты, использованные для разработки и проверки модели;
- описание программы, описание модели и краткое описание математических алгоритмов, задействованных для построения модели;
- системные характеристики оборудования, задействованного для разработки и тренировки модели.
- руководство оператора, программиста и системного программиста.

5. Средства и порядок испытаний

Предварительные испытания проводятся на графическом кластере ВКИ НГУ с использованием графических ускорителей NVIDIA Tesla V100.

Приемо-сдаточные испытания проводятся на кластере Ростелеком с использованием графических ускорителей NVIDIA Tesla T4, 2 процессора Intel(R) Xeon(R) Silver 4210 CPU, ОЗУ 64Гб.

Программные средства для каждого детектора должны быть оформлены в виде отдельного репозитория на графическом кластере. Репозиторий включает размеченные тренировочный, валидационный (тестовый при необходимости) датасеты, скрипты для тренировки моделей, скрипты для вычисления метрик моделей на валидационном и/или тестовом датасете, скрипты для демонстрационного инференса моделей на RTSP потоках заказчика в течение 15-секундного интервала.

В процессе инференса должны быть получены:

- результаты в формате, определенные Техническим заданием;
- демонстрационный видеоролик с отображением детектируемых объектов и ключевых параметров.

Для демонстрационных моделей, подключенных к веб-интерфейсу вывод результатов должен осуществляться с использованием указанных интерфейсов. Для неподключенных моделей вывод результатов осуществляется в папку results. Название json файла и видео файла должны быть в формате «timestamp.json» и «timestamp.mp4», где timestamp — результат метода time.time() в формате int, соответствующий началу обработки RTSP потока.

Испытания проводятся с использованием средства контейнеризации Docker. Программные пакеты и модули должны быть собраны в Docker контейнере. При работе с типовым Docker контейнером должен быть предъявлен скрипт для автоматической установки необходимых зависимостей.

Системные программные средства, используемые программой: операционная система Ubuntu версии 22+.

6. Методы испытаний системы детектирования событий

Для проведения испытаний моделей оператор должен проделать следующие действия:

Проверить выполнение предусловий:

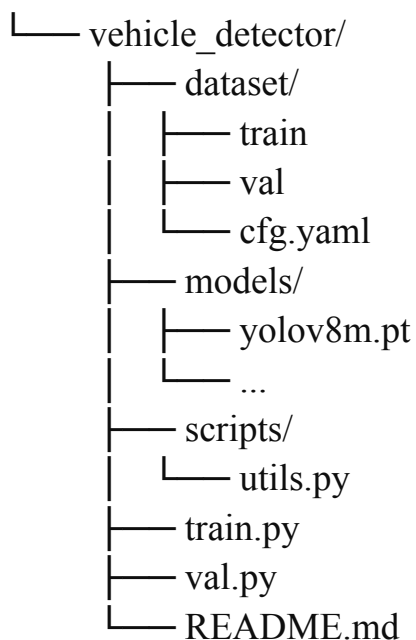
1. Доступ к графическому кластеру ВКИ НГУ с установленными графическими ускорителями NVIDIA Tesla V100 (NVIDIA Tesla T4 для РТК).
2. Наличие программного обеспечения, совместимого с российскими операционными системами, и инструментария машинного обучения, такого как PyTorch и ONNX Runtime.
3. Подготовленные репозитории для каждого детектора, включающие:
 - Размеченные тренировочные, валидационные и тестовые датасеты.
 - Скрипты для тренировки, тестирования, вычисления метрик и инференса моделей.
 - Конфигурационные файлы для запуска моделей.
4. Средства контейнеризации Docker и скрипты для автоматической установки зависимостей.
5. RTSP потоки заказчика для проведения демонстрационного инференса.
6. Установленная операционная система Ubuntu версии 22+.
7. Техническое задание с требованиями к форматам выходных данных.

Проверить окружение:

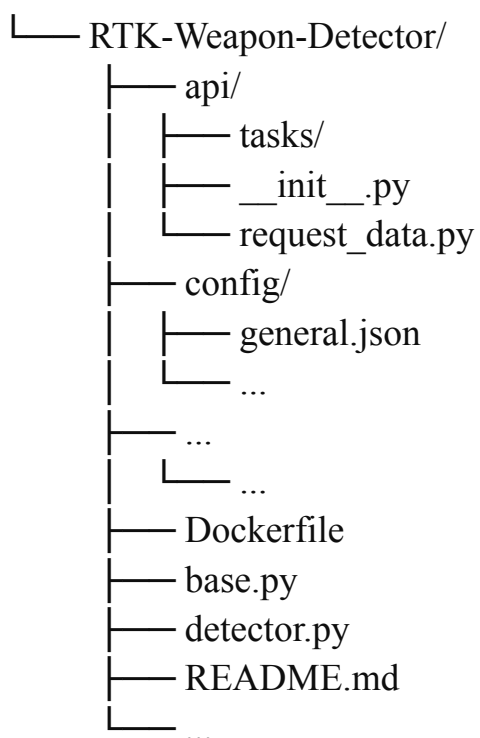
- **Аппаратное обеспечение:** Графический кластер ВКИ НГУ с NVIDIA Tesla V100 (NVIDIA Tesla T4).
- **Программное обеспечение:**
 - ОС Ubuntu 22+.
 - Docker.
 - Библиотеки PyTorch и ONNX Runtime.
- **Входные данные:** RTSP потоки, архивные видеоданные, статические изображения.
- **Выходные данные:**
 - JSON файлы с результатами детекции в формате timestamp.json.

- Видеоролики с наложением детектированных объектов в формате timestamp.mp4.

Структура репозитория с датасетом и обучающими скриптами выглядит следующим образом (пр. детектор транспортных средств):



Структура репозитория для подключения к платформе AP выглядит следующим образом (пр. детектор обнаружения оружия):



Приоритетными считать инструкции приложенные к исходному коду проекта. Последующие шаги предоставляются в ознакомительных целях.

Начать испытания согласно Сценарию:

Шаг 1. Подготовка репозитория

1. Развернуть Docker-контейнер для соответствующего детектора с использованием предоставленного скрипта для работы с платформой AP:

```
docker build -t detector_image ./
docker run --rm -it --name base-detector-api-container --gpus device=0 -p
8000:8000 -v ./logs:/detector-api/logs/ -v
./videos:/detector-api/videos/ detector_image
```

2. Проверить наличие всех необходимых файлов в репозитории:
 - Датасетов (тренировочных, валидационных, тестовых).
 - Скриптов для тренировки, тестирования и инференса.
 - Конфигурационных файлов.

Ожидаемый результат: Все файлы успешно доступны, контейнер запускается и не выдаёт ошибок.

Шаг 2. Тренировка модели

1. Запустить скрипт тренировки модели с указанием тренировочного датасета:

```
python train.py --dataset training_dataset
```

2. Убедиться, что модель сохраняет контрольные точки и генерирует логи метрик (точность, полнота и AP).

Ожидаемый результат: Модель успешно обучена, контрольные точки сохранены, метрики вычислены.

Шаг 3. Тестирование модели на валидационном датасете

1. Выполнить скрипт для вычисления метрик на валидационном датасете:

```
python val.py --dataset validation_dataset
```

2. Сравнить метрики модели с минимальными значениями, указанными в ТЗ:

- Точность (Precision).
- Полнота (Recall).
- Среднее значение точности (AP) на порогах 0.5 и 0.95.

Ожидаемый результат: Метрики соответствуют минимальным значениям ТЗ.

Шаг 4. Демонстрационный инференс на RTSP потоке

1. Запустить инференс на RTSP потоке с ограничением по времени (15 секунд):

```
python infer.py --stream rtsp://example_stream --duration 15
```

2. Убедиться, что результаты инференса сохранены:
 - JSON-файл timestamp.json с информацией о детектированных объектах.
 - Видеофайл timestamp.mp4 с наложением объектов и ключевых параметров.

Ожидаемый результат:

- JSON-файл включает информацию о детектированных объектах, их координатах и классах.
- Видеофайл визуализирует детектированные объекты с наложением данных.

Шаг 5. Проверка интеграции

1. Для моделей с подключением к веб-интерфейсу:
 - Убедиться, что данные передаются корректно и отображаются в указанных интерфейсах.
2. Для остальных моделей:
 - Проверить сохранение результатов в папке results.

Ожидаемый результат: Результаты корректно передаются в интерфейсы или сохраняются в указанной директории.

Общий итоговый Ожидаемый результат

1. Модель корректно обучена и протестирована, результаты соответствуют минимальным требованиям ТЗ.
2. JSON-файлы и видеоролики генерируются в указанном формате.
3. Интеграция с Телеграм-ботом, веб-интерфейсом или файловой системой функционирует без ошибок.
4. Система готова к демонстрации и интеграции в целевую среду.

Смотри таблицу для отчета об испытаниях в следующем разделе.

7. Результаты предварительных испытаний

По результатам разработки для каждой из моделей детектирования вычислены и предоставлены следующие количественные параметры (метрики) модели:

	Ожидаемый результат, версия beta			Фактический результат		
	precision (точность)	recall (полнота)	mAP (0.5-0.95)*	precision (точность)	recall (полнота)	mAP (0.5-0.95)*
Детектор лица человека	0.98	0.95	0.67*	0.982	0.96	0.731
Детектор силуэта человека	0.87	0.83	0.62*	0.926	0.728	0.803
Детектор лежащего человека	0.58	0.64	0.38*	0.633	0.688	0.487
Детектор типа транспортного средства	0.87	0.79	0.86	0.91	0.81	0.87
Детектор дыма	0.94	0.95	0.85	0.98	0.94	0.83
Детектор огня	0.97	0.93	0.85	0.970	0.968	0.827
Детектор оружия	0.87	0.86	0.67*	0.97	0.92	0.71*
Детектор движения в кадре	0.90	0.85	0.90	0.75	0.70	0.75*
Детектор резкого ускорения объекта в кадре	0.96	0.96	0.7	0.96	0.96	0.7
Детектор агрессивного поведения	0.76	0.88	0.37*	0.76	0.88	0.37*
Детектор определения (распознавания) государственных регистрационных знаков	0.86	0.84	0.43*	0.99	0.99	0.95*

Количественные показатели предоставляемых детекторов соответствуют заявленным уровням Технического задания.

8. Сценарии для испытаний детекторов

Рассмотрим сценарии испытаний следующих детекторов:

1. Детектор лица человека
2. Детектор силуэта человека
3. Детектор лежащего человека
4. Детектор типа транспортного средства и определения (распознавания) государственных регистрационных знаков
5. Детектор дыма
6. Детектор огня
7. Детектор оружия
8. Детектор движения в кадре
9. Детектор резкого ускорения объекта в кадре
10. Детектор агрессивного поведения

Приоритетными следует считать инструкции, приложенные к исходному коду проекта каждого детектора. Последующие сценарии предоставляются в ознакомительных целях.

Эти сценарии позволяют провести детальное тестирование каждого детектора с учётом их специфики, обеспечивая полное соответствие функциональности системы заявленным требованиям.

Сценарий 1: Тестирование детектора лица человека

Предусловия

1. Доступ к репозиторию проекта с исходными данными и предобученной сетью YOLO и SCRFD.
2. Видеопоток с изображениями людей различной этнической принадлежности, возраста, пола, а также при разных условиях освещения.
3. Настроенный Docker-контейнер с установленными зависимостями и готовыми скриптами для инференса.

Окружение

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для PTK) для ускорения инференса.
- **Камеры:**

- Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker.
- **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (720x576 для помещений, 1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run --gpus device=0 -p 8000:8000 -v ./logs/:/detector-api/logs/ -it detector_face
```

3. Запуск инференса на видеопотоке и его анализ:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка ключевых этапов обработки:

- Убедиться, что YOLO делит кадры на индивидуальные треки для каждого лица человека.

5. Оценка производительности:

- Проверить значения **precision**, **recall**, и **mAP (0.5-0.95)** на тестовом наборе данных.

6. Валидация сетевого подключения:

- Убедиться, что результаты обработки передаются клиенту без задержки более 5 минут.

Ожидаемый результат

- JSON-выход:

```
{
  "timestamp": 1674487988888,
  "cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
  "recognitionTypeId": "rtk_ngu_srvr",
  "eventType": "face",
  "confidence": 0.723,
  "imageBase64": "string",
  "boxImageBase64": "string",
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  }
}
```

- Модель демонстрирует точность 0.982, полноту 0.96, mAP (0.5-0.95) 0.731, что превышает ожидаемые параметры.
- Видеофайл с визуализацией **bounding box** и ключевых точек для каждого обнаруженного лица.
 - Задержка передачи данных между детекцией и клиентом не превышает 5 минут.

Сценарий 2: Тестирование детектора силуэта человека

Предусловия

1. Доступ к видеопотоку с присутствием людей в различной одежде и с аксессуарами.
2. Настроенный Docker-контейнер с установленными зависимостями.

Окружение

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для РТК) для ускорения инференса.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.

- Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker-контейнер.
- **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архивные видеозаписи с людьми разного возраста, пола, одежды, аксессуаров.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (720x576 для помещений, 1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run -it --gpus device=0 -p 8000:8000 -v  
./logs/./detector-api/logs/ detector_silhouette
```

3. Запуск анализа видеопотока с людьми:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка локализации силуэтов:

- Убедиться, что YOLO корректно локализует силуэты людей на кадрах, возвращая bounding box и числовой показатель вероятности.

5. Проверка атрибутов:

- Убедиться, что трекер SFSORT отслеживает идентичные силуэты на кадрах.
- Проверить, что модель CLIP генерирует корректные атрибуты для каждого обнаруженного силуэта (пол, возраст, головной убор, одежда, аксессуары).

6. Валидация данных:

- Сравнить атрибуты с эталонными значениями из тестового набора.

7. Оценка производительности:

- Проверить метрики **precision**, **recall**, и **mAP (0.5-0.95)** на тестовом наборе данных.

8. Валидация сетевого подключения:

- Убедиться, что результаты обработки передаются клиенту без задержки более 5 минут.

Ожидаемый результат

- JSON-выход:

```
{
  "timestamp": 1674487988888,
  "cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
  "recognitionTypeId": "rtk_ngu_sa",
  "eventType": "person",
  "confidence": 0.723,
  "imageBase64": "string",
  "boxImageBase64": "string",
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  },
  "gender": "мужчина",
  "age": "17-35 лет",
  "headwear": "капюшон",
  "top_color": "черный",
  "bottom_color": "черный",
  "top_type": "толстовка",
  "bottom_type": "брюки",
  "handbag": "да",
  "backpack": "не определен",
  "recognitionAreasSettings": {
    "areas": [
      [[0.01, 0.10], [0.22, 0.35], [0.12, 0.35], [0.22, 0.35],
      [0.22, 0.35]],
      [[0.20, 0.23], [0.81, 0.75], [0.22, 0.35], [0.12, 0.35]]
    ]
  }
}
```

```

    "max_obj_size": null,
    # мы ожидаем от платформы AP значения по максимальному размеру объекта.
    # Изначально его нет, поэтому возможно значение null

    "min_obj_size": 53
  },

  "lineCrossingSettings": {
    "line": [[0.22, 0.73], [0.82, 0.11]],
    "direction": "any", // type: 'any' | 'ab' | 'ba'
    "max_obj_size": null,
    # мы ожидаем от платформы AP значения по максимальному размеру объекта.
    # Изначально его нет, поэтому возможно значение null
    "min_obj_size": 53
  }
}

```

Файл содержит 9 атрибутов:

- gender (пол),
- age (возраст),
- headwear (наличие головного убора),
- top_color (цвет верха одежды),
- bottom_color (цвет низа одежды),
- top_type (тип верха одежды),
- bottom_type (тип низа одежды),
- backpack (наличие рюкзака),
- handbag (наличие сумки в руке).

Уточнены значения для каждого атрибута, соответствующие возможным вариантам их наличия/отсутствия и описанию в видеопотоке.

Метрики модели:

```

{
  "precision": 0.926,
  "recall": 0.728,
  "mAP": 0.803
}

```

Дополнительные результаты:

- Видеофайл с визуализацией bounding box, траекторий движения, и сгенерированных атрибутов.
- Задержка передачи данных между детекцией и клиентом не превышает 5 минут.
- Модель показывает точность 0.926, полноту 0.728, и mAP 0.803, что соответствует минимальным требованиям.

Сценарий 3: Тестирование детектора лежащего человека

Предусловия:

1. Доступ к видеопотокам, содержащим сценарии с неподвижными людьми, такими как человек, лежащий на полу или земле, а также движущиеся люди для проверки классификации.
2. Настроенный Docker-контейнер с установленными зависимостями.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для РТК) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** (для помещений) и **1920x1080** (для улиц).
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами, с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопоток или архивные видео с неподвижными и движущимися людьми.

Сценарий тестирования:

1. **Подготовка оборудования:**
 - Убедиться, что камеры установлены так, чтобы избежать засветки и попадания прямых солнечных лучей на объектив.
 - Проверить корректность видеопотока и настройку разрешения камер.
2. **Запуск Docker-контейнера с моделью YOLOv8:**

```
docker run -it --gpus device=0 -p 8000:8000 -v
```

```
./logs:/detector-api/logs/ detector_prone_person
```

3. Анализ временных интервалов неподвижности:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка ключевых этапов обработки:

- Убедиться, что модель корректно детектирует людей и отслеживает их перемещения.
- Проверить, что неподвижность фиксируется по истечении заданного времени N секунд.
- Оценить точность классификации позы (стоя, сидя, лежа).
- Убедиться, что положение скелета человека корректно идентифицируется, и его неподвижность подтверждается за M секунд.

5. Проверка временных меток и данных:

- Сравнить временные метки неподвижности с реальными значениями из тестового набора.
- Проверить соответствие позы и времени начала/окончания неподвижности.

6. Оценка производительности:

- Проверить метрики precision, recall, и mAP (0.5-0.95) на тестовом наборе данных.

7. Валидация сетевого подключения:

- Убедиться, что результаты обработки передаются клиенту с задержкой не более 5 минут.

Ожидаемый результат:

- JSON-выход:

```
{
  "timestamp": 1674487988888,
  "cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
  "recognitionTypeId": "rtk_ngu_sa",
  "eventType": "fall",
  "confidence": 0.723,
```

```

"imageBase64": "string",
"boxImageBase64": "string",
"box": {
  "left": 0.634375,
  "right": 0.7054688,
  "top": 0.40555555,
  "bottom": 0.44444445
}
"recognitionAreasSettings": {
  "areas": [
    [[0.01, 0.10], [0.22, 0.35], [0.12, 0.35], [0.22, 0.35],
    [0.22, 0.35]],
    [[0.20, 0.23], [0.81, 0.75], [0.22, 0.35], [0.12, 0.35]]
  ],

  "max_obj_size": null,
# мы ожидаем от платформы AP значения по максимальному размеру объекта.
Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
},

"lineCrossingSettings": {
  "line": [[0.22, 0.73], [0.82, 0.11]],
  "direction": "any", // type: 'any' | 'ab' | 'ba'
  "max_obj_size": null,
# мы ожидаем от платформы AP значения по максимальному размеру объекта.
Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
}
}

```

```

{
  "precision": 0.633,
  "recall": 0.688,
  "mAP": 0.487
}

```

Дополнительные результаты:

- Видеофайл с визуализацией скелета, траекторий движения, и классификации поз.

- Убедительное подтверждение неподвижности на протяжении заданного интервала времени.

Описание результатов:

1. Модель демонстрирует **точность 0.633**, **полноту 0.688**, и **mAP 0.487**.
2. Детектор корректно идентифицирует неподвижных людей и классифицирует их позы (стоя, сидя, лежа).
3. Позиция скелета и временные метки фиксируются с точностью, соответствующей реальным значениям.
4. Задержка передачи данных клиенту не превышает 5 минут.

Уточнения:

1. Поддержка классификации поз (стоя, сидя, лежа) и стабильности скелета за заданное время.
2. Учет временных интервалов для фиксации неподвижности.
3. Актуализация метрик и визуализации данных для подтверждения стабильности системы.

Сценарий 4: Тестирование детектора типа транспортного средства совмещенный с распознаванием ГРЗ и определением марки/модели

Предусловия:

1. Доступ к видеопотокам с транспортными средствами 9 различных классов: легковые автомобили, грузовые автомобили, автобусы, велосипеды, мототранспорт, квадроциклы, средства индивидуальной мобильности, специализированный транспорт, строительная техника.
2. Настроенный Docker-контейнер с установленными зависимостями.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для РТК) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**

- RTSP-видеопоток или архивные видео с различными транспортными средствами.

Сценарий тестирования:

1. Подготовка оборудования:

- Убедиться, что камеры установлены так, чтобы избежать засветки и попадания прямых солнечных лучей на объектив.
- Проверить корректность видеопотока и настройку разрешения камер.

2. Запуск Docker-контейнера:

```
docker run -it --gpus device=0 -p 8000:8000 -v
./logs/:/detector-api/logs/ detector_vehicle_type
```

3. Запуск инференса и анализ видеопотока с транспортными средствами:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка работы алгоритма:

- Убедиться, что треки объектов корректно обновляются при движении.
- Проверить точность идентификации транспортных средств на основе класса и местоположения.
- Убедиться, что ГРЗ распознаётся и марки/модели схожи с действительными значениями ТС.

5. Сравнение метрик модели:

- Проверить значения **precision**, **recall**, и **mAP (0.5-0.95)** на тестовом наборе данных для детектора типа транспортных средств.

6. Валидация временных меток:

- Убедиться, что временные метки для обнаруженных объектов совпадают с реальными значениями.

7. Проверка JSON-выхода:

- Убедиться, что выходной JSON включает **vehicleTypeId** – тип транспортного средства, позицию, вероятность и идентификатор.

Ожидаемый результат

- JSON-выход:

```
{
  "timestamp": 1674487988888,
```

```

"cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
"recognitionTypeId": "rtk_ngu_anpr",
"confidence": 0.723,
"imageBase64": "string",
"boxImageBase64": "string",
"box": {
  "left": 0.634375,
  "right": 0.7054688,
  "top": 0.40555555,
  "bottom": 0.44444445
},
"color": "красный",
"vehicleTypeId": 6,
"numberPlate": "A465MH154",
"brand": "Hyundai",
"model": "Solaris"
}

```

Дополнительные результаты:

- Видеофайл с визуализацией bounding box и траекторий движения объектов.
- Корректное отслеживание движущихся транспортных средств.

Уточнения:

1. Поддержка анализа 9 типов транспортных средств - vehicleTypeId (легковые, грузовые, спец. транспорт (скорая, пожарные), строительная техника, мототранспорт, квадроциклы, автобусы, СИМы, велосипеды - идентификационные номера по ГОСТ, в примере цифра 6 означает, что определен тип ТС – легковой автомобиль), что важно для проверки функциональности детектора.
2. Распознавание ГРЗ происходит в соответствии с ТЗ, где размер номерной платы должен быть не менее указанных значений в пикселях.

Сценарий 5: Тестирование детектора дыма

Предусловия:

1. Доступ к видеопотокам или изображениям с наличием дыма различной интенсивности, включая задымление на открытых пространствах и в помещениях, при разном уровне освещения.

2. Репозиторий модели, настроенный для детекции объектов класса “дым”.
3. Настроенный Docker-контейнер с установленными зависимостями.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для РТК) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** (для помещений) и **1920x1080** (для улиц).
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопотоки или архивные видео с задымлением.

Сценарий тестирования:

1. Подготовка оборудования:

- Убедиться, что камеры настроены для корректного захвата видео при разном уровне освещенности.
- Проверить видеопотоки на наличие сцен с дымом и без него.

2. Запуск Docker-контейнера:

```
docker run -it --gpus device=0 -p 8000:8000 -v  
./logs:/detector-api/logs/ detector_smoke
```

3. Анализ видеопотока с задымлением:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка работы трекера:

- Убедиться, что модель корректно идентифицирует зоны дыма и передает данные в трекер.

- Проверить, что трекер присваивает идентификаторы обнаруженным зонам дыма и отслеживает их перемещения между кадрами.
5. **Проверка метрик модели:**
 - Сравнить значения **precision**, **recall**, и **mAP (0.5-0.95)** с требованиями.
 6. **Валидация данных о направлении и скорости распространения дыма:**
 - Убедиться, что трекер корректно определяет направление и скорость распространения дыма.
 7. **Сравнение результатов с эталоном:**
 - Проверить JSON-выход и визуализацию зон дыма на соответствие реальным данным.

Ожидаемый результат

- JSON-выход:

```
{
  "timestamp": 1674487988888,
  "cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
  "recognitionTypeId": "rtk_ngu_srvr",
  "eventType": "smoke",
  "confidence": 0.723,
  "imageBase64": "string",
  "boxImageBase64": "string",
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  }
}
```

Дополнительные результаты:

- Видеофайл с визуализацией зон дыма, ограниченных рамками (bounding box).

Сценарий 6: Тестирование детектора огня

Предусловия:

1. Видеопотоки или изображения с различными типами огня (взрыв, горение жидкости, факел).
2. Репозиторий модели для детекции огня.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопоток или архивные видео с различными транспортными средствами.

Сценарий тестирования

1. Запуск контейнера модели:

```
docker run -it --gpus device=0 -p 8000:8000 -v
./logs/./detector-api/logs/ detector_fire
```

2. Запуск анализа видеопотока:

```
python infer.py --stream rtsp://example_stream --duration 30
```

3. Проверка метрик:

- Сравнить значения precision, recall, и mAP (0.5-0.95).

Ожидаемый результат

- JSON-выход:

```
{
  "timestamp": 1674487988888,
  "cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
  "recognitionTypeId": "rtk_ngu_srvr",
  "eventType": "fire",
```

```

"confidence": 0.723,
"imageBase64": "string",
"boxImageBase64": "string",
"box": {
  "left": 0.634375,
  "right": 0.7054688,
  "top": 0.40555555,
  "bottom": 0.44444445
}
}

```

- Видеофайл с визуализацией детектированных зон огня.

Сценарий 7: Тестирование детектора оружия

Предусловия:

1. Набор видео с присутствием оружия различных типов (пистолет, нож).
2. Репозиторий модели детектора оружия.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для РТК) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопоток или архивные видео с различными транспортными средствами.

Сценарий тестирования:

1. Запуск контейнера модели:

```
docker run -it --gpus device=0 -p 8000:8000 -v
```

```
./logs:/detector-api/logs/ detector_weapon
```

2. Анализ видеопотока:

```
python infer.py --stream rtsp://example_stream --duration 30
```

3. Проверка метрик:

- Убедиться, что precision, recall, и mAP (0.5-0.95) соответствуют или превышают минимальные требования.

Ожидаемый результат

- JSON-выход:

```
{
  "timestamp": 1674487988888,
  "cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
  "recognitionTypeId": "rtk_ngu_srvr",
  "eventType": "weapon",
  "confidence": 0.723,
  "imageBase64": "string",
  "boxImageBase64": "string",
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  }
}
```

- Модель демонстрирует превосходные значения точности и полноты, улучшая mAP по сравнению с минимальными ожиданиями.
- Видеофайл с выделением оружия в кадре.

Сценарий 8: Тестирование детектора движения в кадре

Предусловия:

1. Набор видеопотоков с движущимися объектами различной интенсивности (медленное, быстрое движение).
2. Репозиторий модели для анализа движения.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для PTK) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопоток или архивные видео с различными транспортными средствами.

Сценарий тестирования:

1. Запуск контейнера модели:

```
docker run -it --gpus device=0 -p 8000:8000 -v  
./logs/:/detector-api/logs/ detector_motion
```

2. Анализ движения объектов:

```
python infer.py --stream rtsp://example_stream --duration 30
```

3. Сравнение метрик:
 - Проверить precision, recall, и mAP (0.5-0.95) на тестовом наборе.

Ожидаемый результат:

- JSON-выход:

```
{
```



```

"timestamp": 1674487988888,
"cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
"recognitionTypeId": "rtk_ngu_sa",
"eventType": "motion",
"confidence": 0.723,
"imageBase64": "string",
"boxImageBase64": "string",
"box": {
  "left": 0.634375,
  "right": 0.7054688,
  "top": 0.40555555,
  "bottom": 0.44444445
}
"recognitionAreasSettings": {
  "areas": [
    [[0.01, 0.10], [0.22, 0.35], [0.12, 0.35], [0.22, 0.35],
    [0.22, 0.35]],
    [[0.20, 0.23], [0.81, 0.75], [0.22, 0.35], [0.12, 0.35]]
  ],
  "max_obj_size": null,
  # мы ожидаем от платформы AP значения по максимальному размеру объекта.
  # Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
},
"lineCrossingSettings": {
  "line": [[0.22, 0.73], [0.82, 0.11]],
  "direction": "any", // type: 'any' | 'ab' | 'ba'
  "max_obj_size": null,
  # мы ожидаем от платформы AP значения по максимальному размеру объекта.
  # Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
}
}

```

- Видеофайл с выделением траекторий движения объектов.

Сценарий 9: Тестирование детектора резкого ускорения объекта в кадре

Предусловия:

1. Доступ к видеопотокам с быстрыми движениями объектов (бег, резкое ускорение автомобиля).
2. Репозиторий модели с обученной сетью для анализа ускорения.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для PTK) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопоток или архивные видео с различными транспортными средствами.

Сценарий тестирования:

1. Запуск контейнера модели:

```
docker run -it --gpus device=0 -p 8000:8000 -v
./logs/:/detector-api/logs/ detector_acceleration
```

2. Анализ ускорения объектов:

```
python infer.py --stream rtsp://example_stream --duration 30
```

3. Сравнение метрик:

- Проверить precision, recall, и mAP (0.5-0.95).

Ожидаемый результат:

- JSON-выход:

```
{
```

```

"timestamp": 1674487988888,
"cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
"recognitionTypeId": "rtk_ngu_sa",
"eventType": "boost",
"confidence": 0.723,
"imageBase64": "string",
"boxImageBase64": "string",
"box": {
  "left": 0.634375,
  "right": 0.7054688,
  "top": 0.40555555,
  "bottom": 0.44444445
}
"recognitionAreasSettings": {
  "areas": [
    [[0.01, 0.10], [0.22, 0.35], [0.12, 0.35], [0.22, 0.35],
    [0.22, 0.35]],
    [[0.20, 0.23], [0.81, 0.75], [0.22, 0.35], [0.12, 0.35]]
  ],
  "max_obj_size": null,
  # мы ожидаем от платформы AP значения по максимальному размеру объекта.
  # Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
},
"lineCrossingSettings": {
  "line": [[0.22, 0.73], [0.82, 0.11]],
  "direction": "any", // type: 'any' | 'ab' | 'ba'
  "max_obj_size": null,
  # мы ожидаем от платформы AP значения по максимальному размеру объекта.
  # Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
}
}

```

- Видеофайл с выделением объектов с резким ускорением.

Сценарий 10: Тестирование детектора агрессивного поведения

Предусловия:

1. Доступ к видеозаписям с проявлениями агрессии (драки, агрессивная жестикуляция).
2. Репозиторий модели для анализа поведения.

Окружение:

- **Аппаратные требования:**
 - NVIDIA Tesla V100 (NVIDIA Tesla T4 для PTK) для ускорения инференса.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Docker.
- **Камеры:**
 - Разрешение не менее **720x576** для помещений и **1920x1080** для улиц.
- **Сетевые требования:**
 - Стабильное подключение для передачи данных между серверами и клиентами с задержкой не более 5 минут.
- **Входные данные:**
 - RTSP-видеопоток или архивные видео с различными транспортными средствами.

Сценарий тестирования:

1. Запуск контейнера модели:

```
docker run -it --gpus device=0 -p 8000:8000 -v
./logs/:/detector-api/logs/ detector_aggression
```

2. Анализ поведения:

```
python infer.py --stream rtsp://example_stream --duration 30
```

3. Сравнение метрик:

- Проверить значения precision, recall, и mAP (0.5-0.95).

Ожидаемый результат:

- JSON-выход:

```
{
```

```

"timestamp": 1674487988888,
"cameraId": "222b6be3-a415-4ae2-a473-e850514b3c10",
"recognitionTypeId": "rtk_ngu_sa",
"eventType": "aggressive",
"confidence": 0.723,
"imageBase64": "string",
"boxImageBase64": "string",
"box": {
  "left": 0.634375,
  "right": 0.7054688,
  "top": 0.40555555,
  "bottom": 0.44444445
}
"recognitionAreasSettings": {
  "areas": [
    [[0.01, 0.10], [0.22, 0.35], [0.12, 0.35], [0.22, 0.35],
    [0.22, 0.35]],
    [[0.20, 0.23], [0.81, 0.75], [0.22, 0.35], [0.12, 0.35]]
  ],
  "max_obj_size": null,
  # мы ожидаем от платформы AP значения по максимальному размеру объекта.
  # Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
},
"lineCrossingSettings": {
  "line": [[0.22, 0.73], [0.82, 0.11]],
  "direction": "any", // type: 'any' | 'ab' | 'ba'
  "max_obj_size": null,
  # мы ожидаем от платформы AP значения по максимальному размеру объекта.
  # Изначально его нет, поэтому возможно значение null
  "min_obj_size": 53
}
}

```

- Видеофайл с визуализацией поведения и уровня агрессии.